ARQUIVOS DE I/O

Até agora, tudo que programamos armazenou informações na memória. Ou seja, uma vez que o programa é encerrado, todas as informações coletadas do usuário ou geradas pelo programa são perdidas.

File I/O é a habilidade de um programa de receber um arquivo como entrada ou criar um arquivo como saída.

Para começar, na janela do terminal, digite o código nomes.py e o código é o seguinte:

```
name = input("What's your name?" )
print(f"hello, {name}")
```

Note que ao executar este código, a saída desejada é obtida. O usuário pode inserir um nome. A saída é conforme esperado.

No entanto, e se quisermos permitir que vários nomes sejam inseridos? Como poderíamos fazer isso? Lembre-se de que uma lista é uma estrutura de dados que nos permite armazenar múltiplos valores em uma única variável. O código é o seguinte:

```
names = []

for _ in range(3):
    name = input("What's your name?" )
    names.append(name)
```

Note que o usuário será solicitado três vezes para inserir um nome. O método append é usado para adicionar o nome à nossa lista de nomes.

Este código poderia ser simplificado para o seguinte:

```
names = []

for _ in range(3):
    names.append(input("What's your name?" ))
```

Note que isso tem o mesmo resultado que o bloco de código anterior.

Agora, vamos habilitar a capacidade de imprimir a lista de nomes como uma lista ordenada. O código é o seguinte:

```
names = []

for _ in range(3):
    names.append(input("What's your name?" ))

for name in sorted(names):
    print(f"hello, {name}")
```

Note que uma vez que este programa é executado, todas as informações são perdidas. File I/O permite que seu programa armazene essas informações de forma que elas possam ser usadas posteriormente.

Você pode aprender mais na documentação do Python sobre a função sorted.

OPEN

Open é uma funcionalidade incorporada ao Python que permite abrir um arquivo e utilizá-lo em seu programa. A função open permite que você abra um arquivo de modo que você possa ler dele ou escrever nele.

Para mostrar como habilitar a entrada e saída de arquivo em seu programa, vamos retroceder um pouco e o código é o seguinte:

```
name = input("What's your name? ")

file = open("names.txt", "w")
file.write(name)
file.close()
```

Note que a função open abre um arquivo chamado nomes.txt com a escrita habilitada, como indicado pelo "w". O código acima atribui esse arquivo aberto a uma variável chamada arquivo. A linha arquivo.write(nome) escreve o nome no arquivo de texto. A linha seguinte fecha o arquivo.

Testando seu código digitando python nomes.py, você pode inserir um nome e ele será salvo no arquivo de texto. No entanto, se você executar seu programa várias vezes usando nomes diferentes, você perceberá que este programa reescreverá completamente o arquivo names.txt cada vez.

Idealmente, queremos ser capazes de adicionar cada um de nossos nomes ao arquivo. Remova o arquivo de texto existente digitando rm nomes.txt na janela do terminal. Em seguida, modifique seu código da seguinte forma:

```
name = input("What's your name? ")

file = open("names.txt", "a")
file.write(name)
file.close()
```

Note que a única alteração em nosso código é que o "w" foi alterado para "a" para "anexar". Reexecutando este programa várias vezes, você notará que os nomes serão adicionados ao arquivo. No entanto, você notará um novo problema!

Examinando seu arquivo de texto após executar seu programa várias vezes, você notará que os nomes estão juntos. Os nomes são anexados sem nenhum espaço entre eles. Você pode corrigir esse problema. Novamente, remova o arquivo de texto existente digitando rm nomes.txt na janela do terminal. Em seguida, modifique seu código da seguinte forma:

```
name = input("What's your name? ")

file = open("names.txt", "a")
file.write(f"{name}\n")
file.close()
```

Note que a linha com arquivo.write foi modificada para adicionar uma quebra de linha ao final de cada nome.

Este código está funcionando muito bem. No entanto, há maneiras de melhorar este programa. Acontece que é muito fácil esquecer de fechar o arquivo.

Você pode aprender mais na documentação do Python sobre open.

WITH

A palavra-chave with permite que você automatize o fechamento de um arquivo.

Modifique seu código da seguinte forma:

```
name = input("What's your name? ")

with open("names.txt", "a") as file:
    file.write(f"{name}\n")
```

Note que a linha abaixo do with está indentada.

Até este ponto, estivemos escrevendo exclusivamente em um arquivo. E se quisermos ler de um arquivo? Para habilitar essa funcionalidade, modifique seu código da seguinte forma:

```
with open("names.txt", "r") as file:
    lines = file.readlines()

for line in lines:
    print("hello,", line)
```

Note que readlines tem a habilidade especial de ler todas as linhas de um arquivo e armazená-las em um arquivo chamado linhas. Executando seu programa, você notará que a saída é bastante feia. Parece haver múltiplas quebras de linha onde deveria haver apenas uma.

Há muitas abordagens para corrigir este problema. No entanto, aqui está uma maneira simples de corrigir este erro em nosso código:

```
with open("names.txt", "r") as file:
    lines = file.readlines()

for line in lines:
    print("hello,", line.rstrip())
```

Note que rstrip tem o efeito de remover a quebra de linha excedente no final de cada linha.

Ainda assim, este código poderia ser simplificado ainda mais:

```
with open("names.txt", "r") as file:
  for line in file:
    print("hello,", line.rstrip())
```

Note que executando este código, está correto. No entanto, observe que não estamos ordenando os nomes.

Este código poderia ser ainda mais melhorado para permitir a ordenação dos nomes:

```
names = []

with open("names.txt") as file:
  for line in file:
    names.append(line.rstrip())

for name in sorted(names):
  print(f"hello, {name}")
```

Note que nomes é uma lista vazia onde podemos coletar os nomes. Cada nome é adicionado à lista de nomes na memória. Então, cada nome na lista ordenada na memória é impresso. Executando seu código, você verá que os nomes agora estão devidamente ordenados.

E se quisermos a capacidade de armazenar mais do que apenas os nomes dos alunos? E se quisermos armazenar tanto o nome do aluno quanto a casa deles também?

CSV

CSV significa "valores separados por vírgula".

Na janela do terminal, digite code students.csv. Certifique-se de que seu novo arquivo CSV se pareça com o seguinte:

```
Hermoine,Gryffindor
Harry,Gryffindor
Ron,Gryffindor
```

Draco,Slytherin

Vamos criar um novo programa digitando code students.py e o código é o seguinte:

```
with open("students.csv") as file:
  for line in file:
    row = line.rstrip().split(",")
    print(f"{row[0]} is in {row[1]}")
```

Note que rstrip remove o final de cada linha em nosso arquivo CSV. split diz ao compilador onde encontrar o final de cada um dos nossos valores em nosso arquivo CSV. linha[0] é o primeiro elemento em cada linha do nosso arquivo CSV. linha[1] é o segundo elemento em cada linha em nosso arquivo CSV.

O código acima é eficaz para dividir cada linha ou "registro" do nosso arquivo CSV. No entanto, é um pouco críptico de se olhar se você não estiver familiarizado com este tipo de sintaxe. O Python tem a capacidade integrada que poderia simplificar ainda mais este código. Modifique seu código da seguinte forma:

```
with open("students.csv") as file:
  for line in file:
    name, house = line.rstrip().split(",")
    print(f"{name} is in {house}")
```

Note que a função split na verdade retorna dois valores: o que está antes da vírgula e o que está após a vírgula. Portanto, podemos contar com essa funcionalidade para atribuir dois variáveis de uma vez em vez de uma!

Imagine que gostaríamos novamente de fornecer esta lista como saída ordenada? Você pode modificar seu código da seguinte forma:

```
students = []

with open("students.csv") as file:
  for line in file:
    name, house = line.rstrip().split(",")
```

```
    students.append(f"{name} is in {house}")

for student in sorted(students):
  print(student)
```

Note que criamos uma lista chamada alunos. Adicionamos cada string a esta lista. Então, nós produzimos uma versão ordenada de nossa lista.

Lembre-se de que o Python permite dicionários onde uma chave pode ser associada a um valor. Este código poderia ser ainda mais melhorado:

```
students = []

with open("students.csv") as file:
  for line in file:
    name, house = line.rstrip().split(",")
    student = {}
    student["name"] = name
    student["house"] = house
    students.append(student)

for student in students:
  print(f"{student['name']} is in {student['house']}")
```

Note que criamos um dicionário vazio chamado aluno. Adicionamos os valores para cada aluno, incluindo seu nome e casa no dicionário do aluno. Então, nós adicionamos esse aluno à lista chamada alunos.

Podemos melhorar nosso código para ilustrar isso da seguinte maneira:

```
students = []

with open("students.csv") as file:
  for line in file:
    name, house = line.rstrip().split(",")
```

```
    student = {"name": name, "house": house}
    students.append(student)

for student in students:
  print(f"{student['name']} is in {student['house']}")
```

Note que isso produz o resultado desejado, menos a ordenação dos alunos.

Infelizmente, não podemos classificar os alunos como antes porque cada aluno é agora um dicionário dentro de uma lista. Seria útil se o Python pudesse classificar a lista de alunos de dicionários que classifica esta lista de dicionários pelo nome do aluno.

Para implementar isso em nosso código, faça as seguintes alterações:

```
students = []

with open("students.csv") as file:
  for line in file:
    name, house = line.rstrip().split(",")
    students.append({"name": name, "house": house})

def get_name(student):
  return student["name"]

for student in sorted(students, key=get_name):
  print(f"{student['name']} is in {student['house']}")
```

Note que sorted precisa saber como obter a chave de cada aluno. O Python permite um parâmetro chamado key onde podemos definir em que "chave" a lista de alunos será classificada. Portanto, a função get_name simplesmente retorna a chave de aluno["nome"]. Executando este programa, você verá agora que a lista está agora classificada por nome.

Ainda assim, nosso código pode ser ainda mais melhorado. Acontece que se você só vai usar uma função como get_name uma vez, você pode simplificar seu código da maneira apresentada abaixo. Modifique seu código da seguinte forma:

```
students = []

with open("students.csv") as file:
    for line in file:
        name, house = line.rstrip().split(",")
        students.append({"name": name, "house": house})

for student in sorted(students, key=lambda student: student["name"]):
    print(f"{student['name']} is in {student['house']}")
```

Note como usamos uma função lambda, uma função anônima, que diz "Ei Python, aqui está uma função que não tem nome: Dado um aluno, acesse seu nome e retorne isso para a chave.

Infelizmente, nosso código é um pouco frágil. Suponha que mudássemos nosso arquivo CSV para indicar onde cada aluno cresceu. Qual seria o impacto disso em nosso programa? Primeiro, modifique seu arquivo students.csv da seguinte forma:

```
Harry,"Number Four, Privet Drive"
Ron,The Burrow
Draco,Malfoy Manor
```

Observe como, agora, ao executar nosso programa, ele produzirá vários erros.

Agora que estamos lidando com casas em vez de casas, modifique seu código da seguinte forma:

```
students = []

with open("students.csv") as file:
```

```
    for line in file:
        name, home = line.rstrip().split(",")
        students.append({"name": name, "home": home})

for student in sorted(students, key=lambda student: student["name"]):
    print(f"{student['name']} is in {student['home']}")
```

Observe que a execução de nosso programa ainda não funciona corretamente. Você pode adivinhar por quê?

O erro ValueError: too many values to unpack produzido pelo compilador é resultado do fato de que anteriormente criamos este programa esperando que o arquivo CSV seja dividido usando uma "," (vírgula). Poderíamos gastar mais tempo para resolver este problema, mas de fato alguém já desenvolveu uma maneira de "analisar" (ou seja, ler) arquivos CSV!

A biblioteca csv incorporada do Python vem com um objeto chamado leitor. Como o nome sugere, podemos usar um leitor para ler nosso arquivo CSV, apesar da vírgula extra em "Number Four, Privet Drive". Um leitor funciona em um loop for, onde cada iteração o leitor nos dá outra linha de nosso arquivo CSV. Essa linha em si é uma lista, onde cada valor na lista corresponde a um elemento nessa linha. row[0], por exemplo, é o primeiro elemento da linha fornecida, enquanto row[1] é o segundo elemento.

```
import csv

students = []

with open("students.csv") as file:
    reader = csv.reader(file)
    for row in reader:
        students.append({"name": row[0], "home": row[1]})

for student in sorted(students, key=lambda student: student["name"]):
    print(f"{student['name']} is from {student['home']}")
```

Observe que nosso programa agora funciona como esperado.

Até este ponto, temos confiado em nosso programa para decidir especificamente quais partes de nosso arquivo CSV são os nomes e quais partes são as casas. É um design melhor, porém, incorporar isso diretamente em nosso arquivo CSV, editando-o da seguinte forma:

```
name,home
Harry,"Number Four, Privet Drive"
Ron,The Burrow
Draco,Malfoy Manor
```

Podemos modificar nosso código para usar uma parte da biblioteca csv chamada DictReader para tratar nosso arquivo CSV com ainda mais flexibilidade:

```
import csv

students = []

with open("students.csv") as file:
    reader = csv.DictReader(file)
    for row in reader:
        students.append({"name": row["name"], "home": row["home"]})

for student in sorted(students, key=lambda student: student["name"]):
    print(f"{student['name']} is in {student['home']}")
```

Observe que substituímos leitor por DictReader, que retorna um dicionário de cada vez. Além disso, observe que o compilador acessará diretamente o dicionário da linha, obtendo o nome e a casa de cada aluno. Isso é um exemplo de programação defensiva. Contanto que a pessoa que projetou o arquivo CSV tenha inserido as informações do cabeçalho corretamente na primeira linha, podemos acessar essas informações usando nosso programa.

Até este ponto, estivemos lendo arquivos CSV. E se quisermos escrever em um arquivo CSV?

Para começar, vamos limpar nossos arquivos um pouco. Primeiro, exclua o arquivo students.csv digitando rm students.csv na janela do terminal. Este comando só funcionará se você estiver na mesma pasta que seu arquivo students.csv. Em seguida, em students.py, modifique seu código da seguinte forma:

```
import csv


name = input("What's your name? ")

home = input("Where's your home? ")


with open("students.csv", "a") as file:

    writer = csv.DictWriter(file, fieldnames=["name", "home"])

    writer.writerow({"name": name, "home": home})
```

Observe como estamos aproveitando a funcionalidade incorporada do DictWriter, que recebe dois parâmetros: o arquivo que está sendo escrito e os nomes dos campos a serem escritos. Além disso, observe como a função writerow recebe um dicionário como parâmetro. Literalmente, estamos dizendo ao compilador para escrever uma linha com dois campos chamados nome e casa.

Note que há muitos tipos de arquivos dos quais você pode ler e escrever.

Você pode aprender mais na documentação do Python sobre CSV.

ARQUIVOS BINÁRIOS E PIL

Um outro tipo de arquivo que discutiremos hoje é um arquivo binário. Um arquivo binário é simplesmente uma coleção de uns e zeros. Este tipo de arquivo pode armazenar qualquer coisa, incluindo dados de música e imagem.

Existe uma biblioteca Python popular chamada PIL que funciona bem com arquivos de imagem.

GIFs animados são um tipo popular de arquivo de imagem que tem muitos arquivos de imagem dentro dele que são reproduzidos em sequência repetidamente, criando uma animação ou efeito de vídeo simplista.

Imagine que temos uma série de fantasias, como ilustrado abaixo. Aqui está costume1.gif. Gato número 1.

Aqui está outro chamado costume2.gif. Observe como as posições das pernas são ligeiramente diferentes.

Para começar, na janela do terminal, digite code costumes.py. O código é o seguinte:

```
import sys

from PIL import Image

images = []

for arg in sys.argv[1:]:
    image = Image.open(arg)
    images.append(image)
```

```
images[0].save(
    "costumes.gif", save_all=True, append_images=[images[1]], duration=200, loop=0
)
```

Observe que importamos a funcionalidade Imagem do PIL. Observe que o primeiro loop for simplesmente percorre as imagens fornecidas como argumentos de linha de comando e armazena o tema na lista chamada imagens. O 1: começa a fatiar argv em seu segundo elemento. As últimas linhas de código salvam a primeira imagem e também anexa uma segunda imagem a ela, criando um gif animado. Digitando python costumes.py costume1.gif costume2.gif no terminal. Agora, digite o código costumes.gif na janela do terminal e você poderá ver um GIF animado.

RESUMINDO

Agora, não vimos apenas que podemos escrever e ler arquivos textualmente – também podemos ler e escrever arquivos usando uns e zeros. Mal podemos esperar para ver o que vocês conseguirão com essas novas habilidades a seguir.

- E/S de arquivo
- open
- with
- CSV
- PIL